Checking Reasoning Steps Off the Critical Path: Asynchronous Solver Verification and KV-Cache Invalidation for Small Language Models

Aditya Bankoti
University of Washington
Technical report, October 2026

Abstract. Small language models make errors in individual reasoning steps, and an external checker can catch many of them, but checking each step before the next one is written stalls generation. We study a pipeline in which a model writes each reasoning step as a first-order-logic formula with declared dependencies, an SMT solver (Z3) checks the step against the problem's premises, and refuted steps are removed. We ask whether the check improves accuracy, how to keep checking off the critical path, and whether a refuted step and its dependents can be removed from the KV cache in place rather than by truncating everything written after it. On ProverQA-Hard with Qwen3 models of 8B, 14B and 30B-A3B parameters (n = 100 problems for the larger two) the checker does not improve accuracy at any scale; the step format alone adds 8 to 14 points over chain-of-thought. Asynchronous checking, where the model keeps generating while the solver works, is 12 to 67% faster than blocking on every model at no accuracy cost, with the gap growing with checker latency. In-place masking of a refuted step and its declared dependents costs no accuracy at 14B and 30B and is 4 to 10% faster than truncation at high latency, but collapses at 8B (36.7% vs. 63.3%); we trace the collapse to stale cache entries and to the model giving up, not to undeclared dependencies. Finally, a loop detector that crops repeated steps removes 19% of generated tokens and 16% of time with no accuracy change, whereas dependency-scoped waiting and a mask/truncate hybrid add nothing. All runs, logs and the harness are in the project repository.

1 Introduction

Language models in the 8B to 30B range solve multi-step logical reasoning problems unreliably: individual steps are wrong, and later steps build on them. When each step is written as a logical formula, an automated reasoner can decide whether the step follows from the premises, so step-level errors are detectable in principle. Prior work checks steps after the fact to rerank or diagnose a finished chain [2, 4], or online, regenerating from the last verified step when a check fails [3].

Two costs make the online version expensive. First, a blocking check adds the checker's latency to every step. Z3 answers in milliseconds on the formulas studied here, but a realistic checker, an autoformalizer sharing the GPU or a proof assistant, takes seconds. Second, the standard remedy for a refuted step is to truncate the context back to that step and regenerate, discarding every later step including those that never depended on it. An attractive alternative is to remove only the refuted step and its dependents from the KV cache and keep the rest, but whether in-place removal from the cache actually removes the step's influence is contested [13, 14, 15].

This report evaluates that pipeline end to end. Each step carries an explicit dependency list, so the steps of a problem form a DAG; checking runs asynchronously while generation continues; on a refutation the step and its DAG descendants are removed from the live KV cache by one of three policies, truncation, masking, or masking followed by recomputation of the kept steps. Checker latency is simulated on a token clock so that policies can be compared at latencies of 1, 3 and 8 seconds per check. We then diagnose the one regime where masking fails and test three cheap efficiency fixes.

The findings, over five rounds of experiments:

1. **Step checking does not improve accuracy at 8B, 14B or 30B.** The structured step format is what helps, by 8 to 14 points over chain-of-thought (Section 5.1).
2. **Asynchronous checking is 12 to 67% faster than blocking** on every model, at no accuracy cost, with the gap growing with latency (Section 5.2).

3. **DAG-scoped masking is a modest, scale-dependent win over truncation.** At 14B and 30B it costs no accuracy and saves 4 to 10% of time at high latency, significantly fewer tokens at 30B at every latency. At 8B it loses 27 points at 8 s latency (Section 5.3).
4. **The 8B failure is not caused by undeclared dependencies.** A solver-based test finds hidden logical dependents in 2 to 9% of problems, and masking them changes no answer. The loss comes from stale cache entries, which recomputation recovers, and from the model abandoning the proof after a mask (Section 5.4).
5. **Of three efficiency fixes, only loop cropping pays:** −19% tokens and −16% modeled time at 14B with identical answers. Dependency-scoped waiting saves about one second per problem, and a mask/truncate hybrid degenerates into truncation (Section 5.6).

2 Background and related work

Solver-checked reasoning steps. VeriCoT [2] translates each step of a finished chain into SMT-LIB and checks it with Z3 against a growing premise set; Safe [4] does the same with Lean 4 to rerank samples. LogicTrack [3] is the closest correctness loop to ours: it checks each step with Z3 during generation and, on failure, re-prompts from the verified prefix with feedback, reporting a gain from 0.43 to 0.58 for Llama-3.1-8B on its benchmark. LogicGuide [5] instead constrains decoding so that steps are sound by construction. None of these study checker latency, dependency-scoped invalidation, or the KV cache. Whole-problem formalization, where the model translates the entire problem and a solver answers it [20], is the natural upper bound on what a checker can do, and its failure mode, unfaithful translation, is also ours (Section 6).

Step dependency graphs. PARC [6] builds a premise DAG over reasoning steps and defines accumulation errors, steps that are locally valid but rest on a bad premise; Graph of Verification [7] checks claims in topological order of such a DAG. We use the DAG for a different purpose, to decide which later steps a refutation invalidates, and we take it from the model's own `[uses]` declarations. PARC reports that 7B models declare only about 64% of the premises they use, which motivated our solver-based test for undeclared dependencies (Section 5.4).

Removing reasoning steps from the KV cache. Several systems evict reasoning tokens during decoding to save memory: Think Clearly [9] evicts low-attention chunks, PENCIL [10] erases finished thoughts under a learned rule, StepKV [11] evicts whole steps by utility under a budget, and ArborKV [12] manages the cache for tree-of-thoughts search with lazy rehydration on backtracking. Thinking in Place [8] is closest in mechanism: the model marks steps dead and the runtime evicts their KV pages, trained on step-dependence DAGs. All of these are triggered by a budget or a learned signal; ours is triggered by a solver verdict, and the invalidated set is the transitive closure of declared dependencies.

Does masking remove influence? Recent work argues that masking a span is not the same as never having seen it. Positions Are Not Facts [13] finds that answers still match a masked record 74 to 82% of the time, against 34 to 60% after delete-and-recompute; Aborted but Not Forgotten [14] shows a rolled-back branch leaking through retained KV entries; KVEraser [15] notes that exact erasure requires recomputing every later token. CacheBlend [16] recomputes only high-deviation tokens. Our truncate / mask / refresh policies span this space, and our results at 8B reproduce the leakage concern while our results at 14B and 30B show it can be negligible for the kept, independent steps.

Truth maintenance and provenance. The idea that a retracted belief should retract everything derived from it is classical: dependency-directed backtracking [18], justification-based truth maintenance [17], assumption-based TMS [19], and incremental view maintenance by deletion and rederivation [21]. PACMI [22] applies provenance-cascading invalidation to an LLM agent's external memory. Our contribution is to apply the same discipline to the live KV cache, where invalidation has a cost model (regenerate, mask or recompute) that these systems do not need.

Benchmark. ProverQA [1] generates first-order-logic problems with a symbolic prover; each item ships with gold formulas for its premises and conclusion, which is what makes per-step checking against gold premises possible. Its hard split has 6 to 9 reasoning steps, distractor premises, and three labels (True / False / Uncertain).

3 Method

3.1 Step format and checker

The prompt lists the problem's premises, each with its gold first-order-logic formula, and asks the model to reason in numbered steps of the form

```
Step k [uses P2, S1]: ⟨claim in English⟩
FOL: ⟨formula⟩
```

followed by a final `Answer: A/B/C` line. `P2` names a premise and `S1` an earlier step, so the `[uses]` lists define a DAG over the steps of a problem. Formulas use predicates and constants from the problem, the connectives $\neg \wedge \vee \rightarrow \leftrightarrow \oplus$, and the quantifiers $\forall \exists$.

The checker parses the formula φ and asks Z3 two questions against the premise set Γ: whether $\Gamma \vdash \varphi$ (**entailed**) and whether $\Gamma \vdash \neg\varphi$ (**refuted**); otherwise the verdict is **unknown**. Only refuted steps are rejected. Rejecting every non-entailed step, the policy of our first experiments, cost 28% more tokens, forced 13 of 30 answers, and changed no accuracy, so refute-only is the policy throughout. A rejected step is replaced in the transcript by a one-line note (`Step k was refuted: it contradicts the premises`), and a format guard prevents the model from copying such notes as if they were steps. Each problem is capped at 20 steps, 2,500 generated tokens and 8 rejections; after that the answer is forced to the most likely of A, B, C.

3.2 Asynchronous checking and the latency model

In the **sync** condition the model waits for each verdict before writing the next step. In the **async** conditions it keeps generating; a verdict is applied when it arrives, usually one or two steps later (Figure 1), and the final answer waits for any outstanding verdicts.

Checker latency $L \in \{1, 3, 8\}$ s is simulated on a **token clock**: a verdict becomes available after $L/0.0724$ generated tokens, so $L = 1$ s is about 14 tokens. The clock rate, 72.4 ms per generated token, and a fixed cost of 8.6 s per problem (prefill and Z3) were measured on the 8B model and are held fixed for all models, so **modeled time**

$$T = 8.6 \text{ s} + 0.0724 \text{ s} \times \text{generated tokens} + \text{checker waits}$$

is a function of the transcript alone and comparable across machines and models. Raw wall time on the GPU is reported separately where it matters.

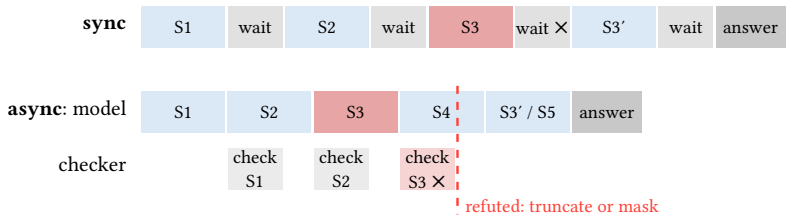


Figure 1: Blocking versus asynchronous checking. With sync, every step waits L for its verdict. Async keeps generating while the solver works and acts on a refutation when it arrives (here, during S4).

3.3 Invalidation policies

When a verdict of **refuted** arrives for step S_k, the **invalidated set** is S_k together with every later step whose `[uses]` chain reaches S_k. Three policies remove it (Figure 2):

- **truncate** (`lag_trunc`): cut the KV cache back to the start of S_k and regenerate everything after it. Exact by construction; discards independent later steps too.
- **mask** (`lag_mask`): remove exactly the invalidated steps' token ranges from the cache and continue generating after the last kept step. Independent later steps are kept as they are.
- **refresh** (`lag_refresh`): mask, then recompute the kept later steps so that their cache entries never attended to the refuted step. Exact, at the cost of a prefill over the kept tokens.

Masking is implemented with llama.cpp's `kv_cache_seq_rm` on the step's position range. For every later query this is equivalent to setting the attention logits of those positions to $-\infty$. Positions are not shifted, so the kept steps keep their rotary positions and the cache has gaps; removing a middle range is safe in llama.cpp, whereas a gap at the tail breaks decoding, which is why trailing removals are implemented as truncation. After three consecutive rejections the harness backs up to the last accepted step.

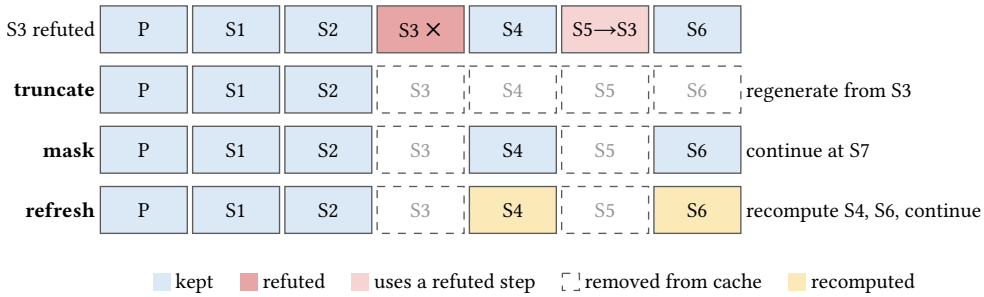


Figure 2: What each policy does to the KV cache when step S3 is refuted. S5 declared `[uses S3]`, so it is invalidated too, while the independent steps S4 and S6 stay under mask and refresh. Truncation discards all of them.

3.4 Efficiency fixes

Three fixes, evaluated in Section 5.6, target the remaining waste in the asynchronous pipeline (Figure 3):

- **Dependency-scoped waiting** (`DEPWAIT`). At answer time the pipeline waits only for the verdicts of the last live step and its transitive `[uses]` ancestors. Pending verdicts on steps the answer does not depend on are not waited for.
- **Loop cropping** (`LOOP=2`). If a new step's formula is identical to that of an earlier live step, the new step is cropped from the cache and replaced by a note (`it repeats an earlier step`). After three crops the answer is forced. This targets a failure mode seen in every condition, including the unchecked ones: the model repeating one formula until it hits the step cap (Section 5.5).
- **Mask/truncate hybrid** (`HYBRID=50`). Under refresh, if fewer than 50 tokens of kept steps would survive a mask, truncate instead; recomputation is not worth a prefill for a short suffix.

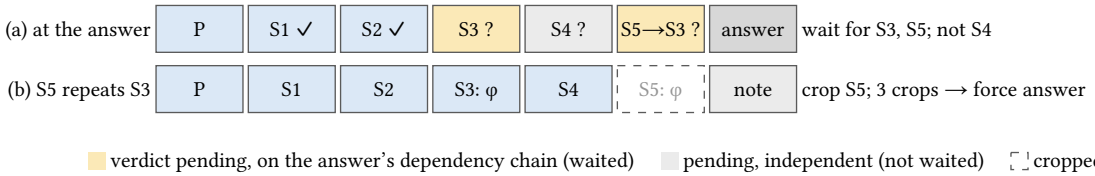


Figure 3: The efficiency fixes. (a) Dependency-scoped waiting: the answer depends on S5, which uses S3, so only those two verdicts are awaited. (b) Loop cropping: a step whose formula repeats an earlier step's is removed and noted.

3.5 Diagnostics for the 8B failure

Two further variants were used only to diagnose why masking loses accuracy at 8B (Section 5.4). **Solver-inferred dependents** (`INFER`) extends the invalidated set with any kept step that the premises alone do not prove but that would follow if the refuted formula were believed. The test starts from the problem's rules plus the refuted formula, adds each ground fact that remains consistent, and checks entailment; starting from the full premise set would prove everything from a refuted formula by *ex falso*. **Forced retry** (`RETRY`) forces the token after a mask to begin a new step, to stop the model from jumping straight to an answer.

4 Experimental setup

Benchmark. ProverQA-Hard [1], the 500-item hard development split: 6 to 9 reasoning steps per problem, distractor premises, labels True / False / Uncertain (A / B / C). Z3 on the gold formulas reproduces the gold label on 498 of 500 items; the two exceptions (ids 411 and 473) are excluded. Always answering Uncertain scores about 40%. Samples are drawn with seed 0: 30 problems for the 8B experiments and 100 for the 14B and 30B. The efficiency experiment (Section 5.6) uses a 30-problem sample of which 28 are in the 100-problem sample; paired comparisons use those 28.

Models. Qwen3-8B, Qwen3-14B and Qwen3-30B-A3B (a mixture of experts with 3B active parameters), all Q4_K_M quantized GGUF files from Qwen's Hugging Face repositories, run with llama.cpp (llama-cpp-python 0.3.36) in non-thinking mode, greedy decoding, temperature 0.7 after a rejection. The 8B latency sweep ran on an Apple M4 (24 GB, Metal); everything else ran on a Colab L4 (24 GB), about 18 GPU-hours in total. The 30B-A3B was chosen as the main model because it structures proofs best (15.3 steps per problem in the unchecked format against 10.1 for the 14B), has the highest checked accuracy, and is about 2.5 times faster than the dense 14B.

Conditions. `base`: ProverQA's own two-shot chain-of-thought prompt. `fmt`: the step format with formulas, unchecked. `sync`: blocking check. `trunc`, `mask`: asynchronous check at $L \in \{1, 3, 8\}$ s. `refresh`: at $L = 8$ s. For the 30B we also ran a **no-cap** ablation (v3): 60-step and 5,000-token caps, the checker verifying every formula with an extra **restated** verdict for formulas that merely repeat a premise, and a verdict cache.

Metrics and statistics. Accuracy; generated tokens per problem; wasted tokens (discarded by truncation, masking or cropping); modeled time (Section 3.2); L4 wall time. All comparisons are paired on the same problems and use two-sided exact sign tests (McNemar's test for accuracy). With $n = 30$ a sign test reaches $p < 0.05$ only at 6 wins to 0 losses, so the 30-problem runs can detect a 15-point accuracy change but not a 5-point one.

5 Results

5.1 Accuracy

Condition	8B (n=30)	14B (n=100)	30B (n=100)	30B, no cap
base (plain CoT)	46.7	58	57	57
fmt (steps, unchecked)	56.7	66	67	71
sync	53.3	65	73	71
trunc, L = 1 / 3 / 8 s	63.3 / 63.3 / 63.3	70 / 70 / 70	72 / 73 / 73	71 / 71 / 71
mask, L = 1 / 3 / 8 s	60.0 / 53.3 / **36.7**	70 / 71 / 71	74 / 72 / 71	76 / 72 / 73
refresh, L = 8 s	53.3	73	71	72

Table 1: Accuracy (% correct). The 8B mask loss at $L = 8$ s is the only significant checked-versus-checked difference (0 wins / 8 losses against trunc, $p = 0.008$).

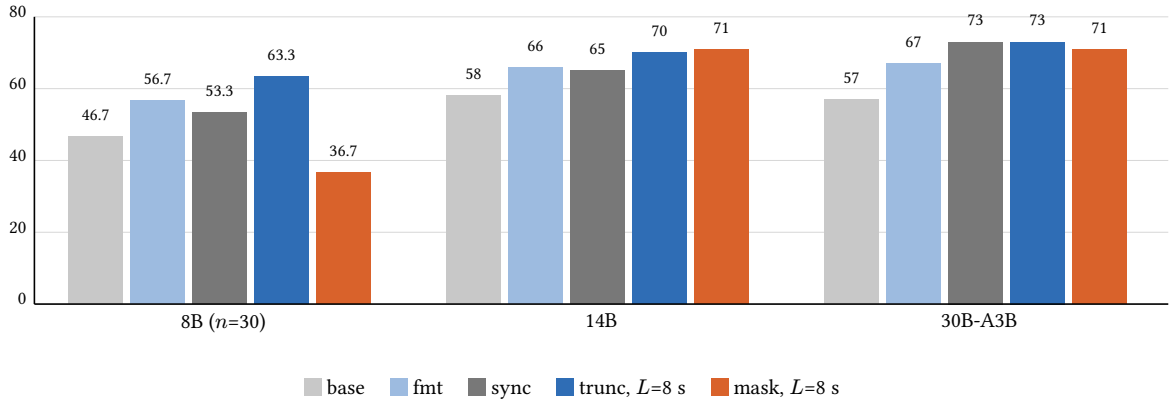


Figure 4: The step format helps; the checker on top of it does not. Masking collapses at 8B with a slow checker but holds at 14B and 30B.

The checker does not improve accuracy at any scale. Checked (sync) minus unchecked (fmt) is −3 points at 8B, −1 at 14B (8 wins / 9 losses, $p = 1.0$), +6 at 30B (8/2, $p = 0.11$) and 0 in the 30B no-cap run (9/9). The asynchronous conditions are no better: the best checked condition at 8B (trunc, 63.3%) beats fmt by 3 wins to 1 loss ($p = 0.62$). What the checker catches are mostly translation errors, malformed formulas or English claims forced into the wrong formula, rather than reasoning errors; at 8B, 58% of rejections were format failures before the format guard, and the checker's feedback biased answers toward Uncertain.

The step format itself helps. fmt beats plain chain-of-thought by 10 points at 8B, 8 at 14B, 10 at 30B and 14 in the no-cap run (27/13, $p = 0.04$). The prompt also shows the formula of every premise, which may account for part of this.

Per class, the checker's only gain is on Uncertain problems. On problems whose gold label is Uncertain, trunc beats fmt 3/0, 6/2 and 3/0 on the three models (12/2 pooled); on True and False problems it does not. Refuting a step that "proves" an unprovable goal is the one case where the solver corrects the model.

5.2 Asynchronous versus blocking checks

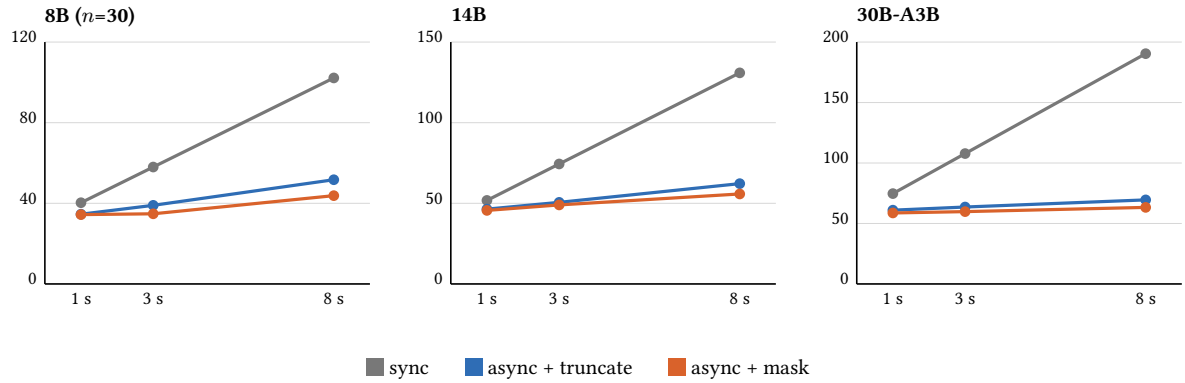


Figure 5: Modeled seconds per problem versus checker latency. Blocking cost grows with every check; asynchronous cost barely moves.

Model	L	sync	trunc	mask	mask vs sync	mask vs trunc	mask fewer tokens, W/L (p)
8B (n=30)	1 s	40.3	34.6	34.4	−15%	−1%	4/3 (1.0)
	3 s	58.0	39.0	34.8	−40%	−11%	7/2 (0.18)
	8 s	102.2	51.7	43.8	−57%	−15%	9/3 (0.15)
14B	1 s	51.8	46.4	45.6	−12%	−2%	10/4 (0.18)
	3 s	74.4	50.6	49.0	−34%	−3%	16/9 (0.23)
	8 s	130.9	62.2	55.8	−57%	−10%	**32/10 (0.001)**
30B-A3B	1 s	74.7	61.0	58.7	−21%	−4%	**21/9 (0.04)**
	3 s	107.8	63.6	59.8	−45%	−6%	**24/9 (0.01)**
	8 s	190.4	69.5	63.3	−67%	−9%	**26/10 (0.01)**

Table 2: Modeled seconds per problem. Last column: problems on which mask generated fewer / more tokens than trunc (exact sign test; bold = $p < 0.05$).

Asynchronous checking is the robust systems result. Against blocking it saves 12 to 67% of modeled time, the saving growing with latency on every model (Figure 5, Table 2), and it gets the same problems right: at 8B, trunc is faster than sync on 27 to 29 of 30 problems at every latency and its accuracy is identical at L = 1, 3 and 8 s. The mechanism is simple: with sync every check adds L to the critical path, whereas an asynchronous verdict only costs time when it is a refutation, and then only the regenerated tokens.

5.3 Masking versus truncation

At 14B and 30B, masking the invalidated steps in place costs no accuracy (every mask-versus-trunc comparison has $p \geq 0.18$; the largest difference is 7 wins / 2 losses for mask at 30B, L = 1 s, no cap) and recomputing the kept steps adds nothing (refresh versus mask 5/3, 4/4, 3/4). It generates significantly fewer tokens than truncation at 30B at every latency and at 14B at L = 8 s (Table 2), which is worth 4 to 10% of modeled time; the saving grows with latency, because a verdict that arrives later has more independent work behind it to keep.

At 8B the picture inverts. Masking is as good as truncation at L = 1 s but loses 10 points at 3 s and 27 points at 8 s, where it wins no problem and loses 8 (p = 0.008). Recomputing the kept steps (refresh) recovers 5 of the 8, which shows that stale cache entries, kept steps whose keys and values were computed while attending to the refuted step, carry a large part of the damage at this scale. A replication of the L = 8 s comparison on a different build (Colab L4, Table 3) found the same direction with a smaller gap (18/30 versus 22/30; 2 wins / 6 losses, p = 0.29).

The saving does not come from a smaller cache. Peak live cache is 2,200 to 2,500 cells per problem under every checked condition, and the masked spans themselves are small (19 to 38 tokens at peak). Masking saves by regenerating less after a refutation.

5.4 Why masking fails at 8B

Two hypotheses were testable on the existing logs. The first is that the model relies on refuted steps without declaring it, so masking keeps steps it should remove (the leakage mechanism of [13, 14]). The solver test of Section 3.5 finds such **hidden dependents** in 2 of 30 problems at 8B, 3 to 5 of 100 at 14B and 6 to 9 of 100 at 30B, and on those problems mask and trunc tie. The second is that the model **quits**: after a mask the 8B wrote no further step in 16 of 20 masked problems and answered Uncertain in 16 of 20 (truncation: 13 and 13), getting 1 of 9 True problems right against 6 of 9.

Condition (8B, L4, L = 8 s, n = 30)	Accuracy	Tokens	Masks / truncations	Answers A/B/C	True right
trunc	22/30	558	0 / 91	11 / 7 / 12	8/9
mask	18/30	364	48 / 4	8 / 6 / 16	5/9
mask + RETRY	19/30	616	80 / 9	11 / 6 / 13	7/9
mask + INFER	18/30	366	48 / 4	8 / 6 / 16	5/9

Table 3: Diagnostic run at 8B. Solver-inferred dependents (INFER) masked 7 extra steps and changed no answer. Forcing a retry after a mask (RETRY) recovers True problems but loops, generating more tokens than truncation.

The run in Table 3 closes both cheap fixes. INFER changed nothing (0 wins / 0 losses against mask): the declared `[uses]` lists are faithful at the logic level, so undeclared dependence is not the mechanism. RETRY recovered the True problems (7 of 9) but only one problem overall (2/1 against mask), at the cost of 80 masks and 616 tokens per problem, more than truncation. What remains is stale cache entries (recoverable by refresh, at a cost) and the small model's tendency to abandon the proof once part of its context disappears. At 8B, truncation is the right policy. The remaining lever is **exact** masking: letting each step attend only to the prompt and its declared parents, so that undeclared reliance and stale entries become impossible by construction; this needs a block-sparse attention mask that llama.cpp does not expose.

5.5 Removing the step cap exposes loops

Raising the caps for the 30B (20 ⟶ 60 steps, 2,500 ⟶ 5,000 tokens) changed no accuracy result (every condition $p \geq 0.39$ against the capped run). It did show what the cap was hiding: 20 to 29% of problems run to the new cap, almost all by repeating one formula, for example `perfectionist(Wyatt) v thinks_carefully(Wyatt)` restated up to step 60, each repeat checking as entailed. Loops occur in the unchecked format too (20 of 100 problems), so the model causes them, not the checker. Forced answers are 52 to 75% correct against 70 to 84% for unforced ones. Mean tokens nearly double (fmt 617 ⟶ 1,179) while the median barely moves (720 ⟶ 767); per condition, about 1,000 to 1,150 steps are **restated** and 1,100 to 1,340 formulas hit the verdict cache. This motivated the loop detector.

5.6 Efficiency fixes

The three fixes of Section 3.4 were run at 14B, L = 8 s, on 30 problems, with DEPWAIT and LOOP enabled in every condition and HYBRID=50 in the refresh condition, and compared with the round-3 runs of the same policies on the 28 shared problems (Table 4, Table 5).

Condition	Accuracy	Gen tokens	Wasted	Wait s	Modeled s	Wall s (L4)
trunc	21/28	644	322	6.9	62.1	25.6
mask	20/28	621	243	6.8	60.3	24.9
refresh	20/28	644	316	7.0	62.2	25.6
trunc + DEPWAIT + LOOP	21/28	521	258	5.8	52.1	20.8
mask + DEPWAIT + LOOP	20/28	504	253	6.4	51.5	20.4
hybrid + DEPWAIT + LOOP	20/28	536	276	5.6	53.0	21.5

Table 4: The efficiency fixes at 14B, L = 8 s, on the 28 paired problems. Top: the round-3 runs without fixes, restricted to the same problems. Hybrid is refresh with HYBRID=50.

Comparison	Accuracy W/L (p)	Fewer tokens W/L (p)	Faster W/L (p)	Modeled time
trunc + fixes vs trunc	1/1 (1.0)	8/2 (0.11)	**11/1 (0.01)**	−10.0 s (−16%)
mask + fixes vs mask	1/1 (1.0)	**10/2 (0.04)**	**12/1 (0.003)**	−8.8 s (−15%)
hybrid + fixes vs refresh	1/1 (1.0)	8/4 (0.39)	11/3 (0.06)	−9.2 s (−15%)
mask vs trunc (both with fixes)	0/1 (1.0)	5/6 (1.0)	4/6 (0.75)	−0.6 s
hybrid vs trunc (both with fixes)	0/1 (1.0)	0/2 (0.5)	0/2 (0.5)	+0.8 s

Table 5: Paired tests for Table 4 (exact sign tests, two-sided; bold = $p < 0.05$).

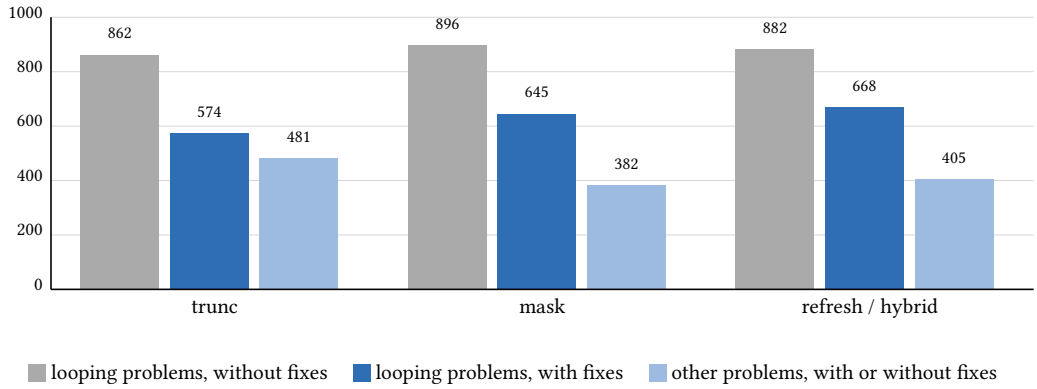


Figure 6: Generated tokens per problem, 14B at $L = 8$ s, split by whether the loop detector fired (12 to 14 of 28 problems). On the other problems the fixed and unfixed runs produced identical transcripts.

The fixes save 15 to 16% of modeled time and 19% of tokens at no detectable accuracy cost, and the saving is significant for trunc and mask. Three observations locate the gain:

- **The whole gain is the loop detector.** The detector fired on 12 to 14 of the 28 problems; on those, tokens fall from about 880 to 570–670 (Figure 6). On the other 14 to 16 problems the transcripts, token counts and modeled times are identical to the decimal: the fixes are inert until a loop or an avoidable wait occurs. Accuracy on the looping problems is the same with and without cropping (8/12, 7/13, 9/14 in both runs), so the cropped tokens were pure waste. Five to six problems per condition reached the three-crop limit and were forced to answer.
- **Dependency-scoped waiting saves about one second per problem.** Waits fall from 6.9 to 5.8 s (trunc) and 6.8 to 6.4 s (mask); only 5 to 6 problems per condition had a pending verdict the answer did not depend on, and all of them were looping problems whose answer was forced. Waiting is about 11% of modeled time at $L = 8$ s, so there was little to recover.
- **The hybrid degenerates into truncation.** The kept-token rule fired 44 times against 3 recomputations (5.8 recomputed tokens per problem): the suffix after a refuted step is usually shorter than 50 tokens. Hybrid and trunc generate nearly identical transcripts (0/2). The threshold is too high, and at 14B recomputation has nothing to recover in any case.

With the fixes in place, mask and trunc remain tied at 14B (5/6 on tokens, 4/6 on time, 0/1 on accuracy).

6 Discussion

What the checker is for. On this benchmark the solver mostly catches translation errors, not reasoning errors, and its one measurable effect on accuracy is on Uncertain problems, where it stops the model from "proving" an unprovable goal. Translation into logic is the bottleneck, as in whole-problem formalization [20, 23]: a formula that parses and is entailed need not say what the English step says. Constrained decoding fixes the syntax but not the faithfulness.

What asynchrony is for. The 12 to 67% saving over blocking is the paper-grade systems result. It requires nothing from the model and loses nothing, and it grows with checker latency, which is exactly the regime (an autoformalizer sharing the GPU, Lean) where a checker is expensive enough to matter.

Where DAG-scoped masking stands. Masking is the proposal's novel mechanism, and after five rounds its value is real but modest: at 14B and 30B it is 4 to 10% faster than truncation at high latency, with no accuracy cost, and it is never better at low latency, where a refutation arrives before much independent work exists. At 8B it is harmful at high latency, and the diagnosis in Section 5.4 says why: not undeclared dependencies, which are rare and inert, but stale entries in

the kept steps plus a model that gives up. Those are both properties of the attention computation, not of the dependency bookkeeping, which is why the next version of the idea should change the attention pattern (each step attends only to the prompt and its declared parents) rather than the invalidation policy. Compared with budget-driven KV eviction [8, 11], our trigger is a correctness signal and our invalidated set is a provenance closure; compared with the leakage literature [13, 14, 15], our results say that leakage is a function of model scale and of how long the stale entries are used.

Loops. The cheapest result in this report is also the most practical: a two-line check for a repeated formula removes a fifth of the tokens at 14B and changes no answer. It applies to the unchecked step format as well.

7 Limitations

- **Simulated latency.** Checker latency is a token clock, not a real slow checker; a real autoformalizer would share the GPU and slow generation itself.
- **One benchmark, gold premises.** ProverQA-Hard ships gold formulas for its premises, so the checker verifies only the model's step translations, an upper bound on what a fully model-translated pipeline would achieve.
- **Sample size.** The 8B latency sweep and the diagnostic and efficiency runs use 30 problems, which detects only large accuracy changes (Section 4). The null accuracy results at $n = 30$ are "no detectable change", not "no change". The 14B and 30B main results use 100 problems.
- **Self-declared dependencies.** The DAG comes from the model's `[uses]` lists. The solver test in Section 5.4 bounds the damage at the logic level but cannot see reliance that is not expressible as entailment.
- **Policy evolution.** Several policies (refute-only rejection, the format guard, the token clock) were fixed in response to measured failures during the first two rounds. The reported runs share one fixed policy and were not tuned on their own results.
- **Quantized models, one decoding setting.** All models are 4-bit, in non-thinking mode, with one seed.

8 Conclusion and next steps

A step-level solver check does not make Qwen3 models of 8B to 30B parameters more accurate on ProverQA-Hard, but running the check asynchronously makes a checked pipeline 12 to 67% cheaper than a blocking one at no cost, and removing a refuted step's dependents from the KV cache in place is a further 4 to 10% at 14B and 30B. A loop detector removes another fifth of the tokens. The clear failure, masking at 8B, is now diagnosed to the attention computation rather than to the dependency graph.

Three next steps follow directly. First, replace the simulated latency with a real slow checker, an LLM autoformalizer sharing the GPU or a Lean backend, and re-run the latency sweep at $n \geq 100$. Second, make masking exact with a block-sparse attention mask in which each step attends only to the prompt and its declared parents; this removes both stale entries and undeclared reliance by construction and is the version of the idea that could beat truncation at 8B. Third, if the accuracy question is to be settled, run the checked-versus-unchecked comparison on several hundred problems with a second model family; the current nulls are informative about effect size (a few points at most) but not conclusive.

Artifacts. Harness: `run.py`, `logic.py`, `analyze.py`, `analyze_eff5.py`; cloud driver: `cloud/colab.sh`, `cloud/vm.sh`; raw runs: `results/v2/` (8B), `results_cloud/Qwen3-14B/`, `results_cloud/Qwen3-30B-A3B/`, `results_cloud/Qwen3-30B-A3B_v3/`, `results_cloud/Qwen3-8B_infer/`, `results_cloud/Qwen3-14B_eff/`; per-round write-ups with all paired tests: `results/RESULTS.md`.

9 References

[1] C. Qi, R. Ma, B. Li, H. Du, B. Hui, J. Wu, Y. Laili and C. He. *Large Language Models Meet Symbolic Provers for Logical Reasoning Evaluation* (ProverQA). ICLR 2025. arXiv:2502.06563.

[2] Y. Feng, N. Weir, K. Bostrom, S. Bayless, D. Cassel, S. Chaudhary, B. Kiesl-Reiter and H. Rangwala. *VeriCoT: Neuro-symbolic Chain-of-Thought Validation via Logical Consistency Checks.* arXiv:2511.04662, 2025.

[3] J. Hu, S. Yang, W. Liu and D. Wang. *LogicTrack: Auditing Reasoning Trajectories of Large Language Models with Formal Logic Solvers.* arXiv:2609.21492, 2026.

[4] C. Liu et al. *Safe: Enhancing Mathematical Reasoning in Large Language Models via Retrospective Step-aware Formal Verification.* ACL 2025. arXiv:2506.04592.

[5] G. Poesia, K. Gandhi, E. Zelikman and N. D. Goodman. *Certified Deductive Reasoning with Language Models* (LogicGuide). arXiv:2306.04031, 2023.
[6] S. Mukherjee, A. Chinta, T. Kim, T. A. Sharma and D. Hakkani-Tür. *Premise-Augmented Reasoning Chains Improve Error Identification in Math Reasoning with LLMs* (PARC). ICML 2025. arXiv:2502.02362.
[7] J. Fang, B. Zhang, C. Wang, J. Wan and Z. Xu. *Graph of Verification: Structured Verification of LLM Reasoning with Directed Acyclic Graphs.* AAAI 2026. arXiv:2506.12509.
[8] T. Jin. *Self-Orchestrating Language Models: Leveraging Semantic Dependence for Efficient Inference* (chapter 4, "Thinking in Place"). arXiv:2609.14850, 2026.
[9] D. Choi et al. *Think Clearly: Improving Reasoning via Redundant Token Pruning.* arXiv:2507.08806, 2025.
[10] C. Yang, N. Srebro, D. McAllester and Z. Li. *PENCIL: Long Thoughts with Short Memory.* ICML 2025. arXiv:2503.14337.
[11] B. Feng et al. *StepKV: Step-Aware KV Cache Compression for LLM Agents.* arXiv:2609.22158, 2026.
[12] Y. Chen, Z. Liu, Z. Huang, R. Gui, H. Wang and L. Liu. *ArborKV: Structure-Aware KV Cache Management for Scaling Tree-based LLM Reasoning.* arXiv:2605.22106, 2026.
[13] C. Zhou et al. *Positions Are Not Facts: The Mismatch Between KV Caches and Memory.* arXiv:2609.33759, 2026.
[14] G. Zhang and H. Yang. *Aborted but Not Forgotten: KV-Cache Retention Breaks Rollback Consistency in Language Agents.* arXiv:2608.15939, 2026.
[15] M. Li et al. *KVEraser: Learning to Steer KV Cache for Efficient Localized Context Erasing.* ICML 2026 Workshop on the Impact of Memorization on Trustworthy Foundation Models. arXiv:2606.17034.
[16] J. Yao et al. *CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion.* EuroSys 2025. arXiv:2405.16444.
[17] J. Doyle. *A Truth Maintenance System.* Artificial Intelligence 12(3), 1979. doi:10.1016/0004-3702(79)90008-0.
[18] R. M. Stallman and G. J. Sussman. *Forward Reasoning and Dependency-Directed Backtracking in a System for Computer-Aided Circuit Analysis.* Artificial Intelligence 9(2), 1977. doi:10.1016/0004-3702(77)90029-7.
[19] J. de Kleer. *An Assumption-Based TMS.* Artificial Intelligence 28(2), 1986. doi:10.1016/0004-3702(86)90080-9.
[20] L. Pan, A. Albalak, X. Wang and W. Y. Wang. *Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning.* Findings of EMNLP 2023. arXiv:2305.12295.
[21] A. Gupta, I. S. Mumick and V. S. Subrahmanian. *Maintaining Views Incrementally.* SIGMOD 1993. doi:10.1145/170035.170066.
[22] Y. Wang et al. *PACMI: Provenance-Aware Cascading Memory Invalidation for Long-Term LLM Agents.* arXiv:2610.05732, 2026.
[23] R. Cornish et al. *FaithformBench: Benchmarking Faithfulness of Mathematical Chain-of-Thought Autoformalisation.* arXiv:2608.10916, 2026.